Name: 宋庭宇 Student ID: 314512091

- **Result：**

- **Source code :**

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def create_atmospheric_degradation(M, N, k=0.002):
    H = np.zeros((M, N))
    for u in range(M):
        for v in range(N):
            exponent = -k * (((u - M / 2) ** 2 + (v - N / 2) ** 2) ** (5 / 6))
            H[u, v] = np.exp(exponent)
    return H

def apply_circular_cutoff(H, radius, M, N):
    H_cutoff = H.copy()
    center_value = H[M // 2, N // 2]

    for u in range(M):
        for v in range(N):
            distance_squared = (u - M // 2) ** 2 + (v - N // 2) ** 2
            if distance_squared > radius ** 2:
                H_cutoff[u, v] = center_value

    return H_cutoff

def apply_inverse_filter(image, degradation_func):
    image_fft = np.fft.fft2(image)
    image_fft_shifted = np.fft.fftshift(image_fft)
    restored_fft = image_fft_shifted / degradation_func
```

```
    restored = np.abs(np.fft.ifft2(np.fft.ifftshift(restored_fft)))

    return restored

def apply_wiener_filter(image, degradation_func, K=0.04):
    image_fft = np.fft.fft2(image)
    image_fft_shifted = np.fft.fftshift(image_fft)
    H_squared = degradation_func ** 2
    wiener_coefficient = H_squared / (H_squared + K)
    restored_fft = (wiener_coefficient / degradation_func) * image_fft_shifted
    restored = np.abs(np.fft.ifft2(np.fft.ifftshift(restored_fft)))
    return restored

def main():
    image = cv2.imread('Fig5.25.jpg', cv2.IMREAD_GRAYSCALE)

    if image is None:
        print("Error: Could not load image 'Fig5.25.jpg'")
        print("Please make sure the image is in the same directory as the script.")
        return

    M, N = image.shape
    print(f"Image loaded: {M}x{N} pixels")
    print("\nCreating atmospheric turbulence degradation model...")
    k_turbulence = 0.002
    H = create_atmospheric_degradation(M, N, k=k_turbulence)
    print("Applying inverse filter with circular cutoff...")
    cutoff_radius = 40
    H_cutoff = apply_circular_cutoff(H, cutoff_radius, M, N)
    restored_inverse = apply_inverse_filter(image, H_cutoff)
    print("Applying Wiener filter...")
    wiener_K = 0.04
    restored_wiener = apply_wiener_filter(image, H, K=wiener_K)
    print("\nDisplaying results...")
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original (Degraded)')
    plt.xticks([])
    plt.yticks([])
    plt.subplot(1, 3, 2)
    plt.imshow(restored_inverse, cmap='gray')
```

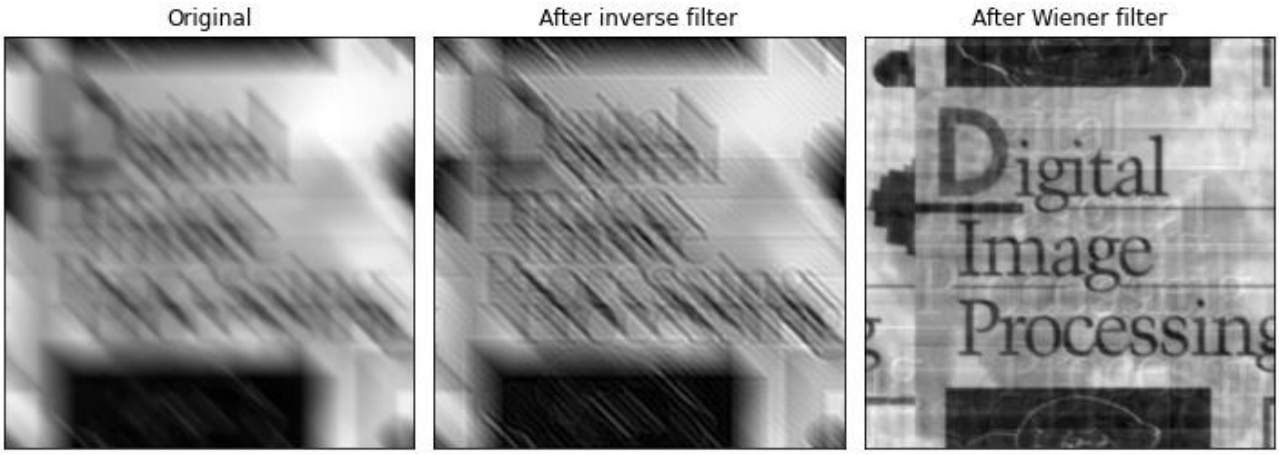
```python
    plt.title(f'Inverse Filter\n(radius={cutoff_radius})')
    plt.xticks([])
    plt.yticks([])
    plt.subplot(1, 3, 3)
    plt.imshow(restored_wiener, cmap='gray')
    plt.title(f'Wiener Filter\n(K={wiener_K})')
    plt.xticks([])
    plt.yticks([])
    plt.tight_layout()
    output_filename = 'Fig5.25_result.png'
    plt.savefig(output_filename, dpi=150, bbox_inches='tight')
    print(f"Result saved as: {output_filename}")

    plt.show()

if __name__ == "__main__":
    main()
```

2.

- **Result：**



- **Source code :**

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def get_butterworth(M, N, cutoff_radius, order):
    B = np.zeros((M, N))
    for u in range(M):
        for v in range(N):
            D = np.sqrt(((u - M / 2) ** 2 + (v - N / 2) ** 2))
            if D == 0 and cutoff_radius == 0:
                B[u, v] = 0
            elif cutoff_radius == 0:
                 B[u, v] = 0
            else:
                B[u, v] = 1 / (1 + (D / cutoff_radius) ** (2 * order))
    return B

def wiener_filter(image, K=0.01):
    img = image.copy()
    M, N = img.shape[:2]
    img = np.array(img, dtype=np.float32)
    fourier = np.fft.fft2(img)
    fourier_shifted = np.fft.fftshift(fourier)
    H = np.zeros((M, N))
    H = get_H(H, M, N)
    H_squared = H ** 2
    epsilon = 1e-10
    wiener_filter_response = H_squared / (H_squared + K + epsilon)
    restore_fourier = (fourier_shifted / (H + epsilon)) * wiener_filter_response
```

```python
    restored_img = np.abs(np.fft.ifft2(np.fft.ifftshift(restore_fourier)))
    return restored_img

def inverse(image):
    img = image.copy()
    M, N = img.shape[:2]
    img = np.array(img, dtype=np.float32)
    fourier = np.fft.fft2(img)
    fourier_shifted = np.fft.fftshift(fourier)
    H = np.zeros((M, N))
    H = get_H(H, M, N)
    cutoff_radius = 60
    order = 10
    B = get_butterworth(M, N, cutoff_radius, order)
    restore_fourier = (fourier_shifted / H) * B

    restored_img = np.abs(np.fft.ifft2(np.fft.ifftshift(restore_fourier)))
    return restored_img

def get_H(H, M, N):
    k = 0.0015
    for u in range(M):
        for v in range(N):
            exponent = -k * (((u - M / 2) ** 2 + (v - N / 2) ** 2) ** (5 / 6))
            H[u, v] = np.exp(exponent)
    return H


def create_motion_blur_psf(image_shape, length, angle):
    M, N = image_shape
    kernel = np.zeros((length, length))
    kernel[int((length - 1) / 2), :] = np.ones(length)
    rotation_matrix = cv2.getRotationMatrix2D(
        (length / 2 - 0.5, length / 2 - 0.5),
        angle,
        1.0
    )
    kernel = cv2.warpAffine(kernel, rotation_matrix, (length, length))
    kernel = kernel / np.sum(kernel)
    psf = np.pad(kernel, ((0, M - length), (0, N - length)), mode='constant')
    return psf

```

```python
def get_atmospheric_degradation(M, N, k=0.0025):
    H = np.zeros((M, N))
    for u in range(M):
        for v in range(N):
            exponent = -k * (((u - M / 2) ** 2 + (v - N / 2) ** 2) ** (5 / 6))
            H[u, v] = np.exp(exponent)
    return H


def apply_inverse_filter(image, degradation_func, cutoff_radius=60):
    M, N = image.shape
    G_fft = np.fft.fft2(image)
    G_shifted = np.fft.fftshift(G_fft)
    butterworth = get_butterworth(M, N, cutoff_radius, order=10)
    epsilon = 1e-10
    restored_fft = (G_shifted / (degradation_func + epsilon)) * butterworth
    restored = np.abs(np.fft.ifft2(np.fft.ifftshift(restored_fft)))

    return restored


def apply_wiener_filter(image, psf, nsr=0.0005):
    G_fft = np.fft.fft2(image)
    H_fft = np.fft.fft2(psf)
    H_conj = np.conj(H_fft)
    H_abs_squared = np.abs(H_fft) ** 2
    wiener_filter_kernel = H_conj / (H_abs_squared + nsr)
    restored_fft = G_fft * wiener_filter_kernel
    restored = np.fft.ifft2(restored_fft).real

    return restored

def main():
    image = cv2.imread('book-cover-blurred.tif', cv2.IMREAD_GRAYSCALE)
    if image is None:
        print("Error: Could not load image 'book-cover-blurred.tif'")
        print("Please make sure the image is in the same directory as the script.")
        return
    M, N = image.shape
    print(f"Image loaded: {M}x{N} pixels")
    print("\nApplying inverse filter with atmospheric degradation model...")
    k_atmospheric = 0.0025
    H_atmospheric = get_atmospheric_degradation(M, N, k=k_atmospheric)
```

```python
    cutoff_radius = 60
    restored_inverse = apply_inverse_filter(image, H_atmospheric, cutoff_radius)
    print("Applying Wiener filter with motion blur PSF...")
    blur_length = 99
    blur_angle = 135  # degrees
    nsr = 0.0005  # Noise-to-Signal Ratio
    psf = create_motion_blur_psf((M, N), blur_length, blur_angle)
    restored_wiener = apply_wiener_filter(image, psf, nsr)
    print("\nDisplaying results...")
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original (Blurred)')
    plt.xticks([])
    plt.yticks([])

    plt.subplot(1, 3, 2)
    plt.imshow(restored_inverse, cmap='gray')
    plt.title(f'Inverse Filter\n(r={cutoff_radius})')
    plt.xticks([])
    plt.yticks([])

    plt.subplot(1, 3, 3)
    plt.imshow(restored_wiener, cmap='gray')
    plt.title(f'Wiener Filter\n(NSR={nsr})')
    plt.xticks([])
    plt.yticks([])

    plt.tight_layout()
    output_filename = 'book-cover-blurred_result.png'
    plt.savefig(output_filename, dpi=150, bbox_inches='tight')
    print(f"Result saved as: {output_filename}")

    plt.show()

if __name__ == "__main__":
    main()
```

3. Please comment and compare your two design filters?

Upon applying both inverse and Wiener filters to 'Fig5.25.jpg,' it was observed that even after parameter tuning, the clarity of the inverse filter output remained inferior to that of the Wiener filter. Consequently, these results demonstrate that the Wiener filter offers superior performance.

Furthermore, regarding the motion-blurred image 'car-moire-pattern.tif,' the inverse filter failed to restore the image despite multiple parameter adjustments, resulting in a persistently blurry output. In contrast, the Wiener filter effectively restored the motion-degraded image, successfully recovering its clarity.